

Recursion

Definition: see Recursion

The actual definition

- Recursion - When a function is called from inside its own code
- Be aware that the joke on the title slide is a problematic example of recursion - it has no stopping condition

Example: Factorials

- $3! = 1*2*3$
- $4! = 1*2*3*4$
 - $= 3! * 4$
- $n! = (n-1)! * n$
- Let's program this in python
- But we need a stopping condition!
- Base case - an input for which a recursive function will not call itself

Example: nested triangles

- Read and run the `nested_triangle.py` code
- Let's move the blue triangle inside the loop for the red triangle, in between the forward and right functions
- How many times does `my_turtle.forward(50)` run? Why?
- Now let's move the green triangle inside the loop for the blue triangle, again in between the forward and right functions
- Before running this, how many green triangles do you expect to see?
- Now, let's do this recursively
- What did we forget?

State Transparency

- Let's add an additional parameter to our nested triangle function: an angle that will be used after the base case but before the for loop
- Make sure that the recursive function call uses this angle
- Now input a small angle (just 1 or 2) when calling the original triangle, and see what happens
- Whenever you write a recursive function, you want to ensure that the function returns everything that you didn't want to change to its original state - this is called state transparency

Example: Tree

- Read and run the `tree.py` code
- Create a `tree3` that calls `tree2` in the same way that `tree2` calls `tree1`
- The difference between `tree2` and `tree3` is tiny, so a single function with an additional parameter would be better
- This will require recursion, but that won't be a problem for us now
- Does this recursive tree work?

Counting segments on a tree

- Make a chart counting the segments on a tree for each level
- Can you find a formula?
- Do you know why this code makes this formula?
- Create a function that counts the segments
- A note on recursive code tracing

Example: Fibonacci numbers

- $F_n = F_{n-1} + F_{n-2}$

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13
F_n	0	1	1	2	3	5	8	13	21	34	55	89	144	233

- In fibonacci.py we have a loop that calculates Fibonacci numbers
- But we can use recursion instead
- Compare the speed of the two methods
- We can speed up the recursive version by remembering which numbers have already been calculated and using the previously calculated value